



MEDAC

Instituto Oficial de Formación Profesional

Sistema de Gestión de Activos e Incidencias

Acceso a Datos

Profesor: Jose Ferrer Grau

Alumnos: Brian Desposorio y
Valentín Ocampo

Fecha de Entrega: 18 de febrero de 2026



1. INTRODUCCIÓN.....	4
2. ARQUITECTURA DEL SISTEMA.....	4
2.1. Organización por Capas.....	4
2.2. Base de Datos.....	5
2.3. Modelo de Datos.....	6
3. FUNCIONALIDADES.....	6
3.1. Gestión de Activos.....	7
3.2. Gestión de Incidencias.....	8
4. EXTRAS IMPLEMENTADOS.....	8
4.1. Extra 6: Sistema de Usuarios y Roles.....	8
4.2. Extra 7: Validaciones Avanzadas.....	9
4.3. Extra 9: Exportación Avanzada.....	10
4.4. Extra 10: Mejoras de Usabilidad.....	11
5. NSTRUCCIONES DE USO.....	12
5.1. Instalación.....	12
5.2. Usuario Administrador.....	13
5.3. Usuario Técnico.....	14
6. CONCLUSIONES.....	15
6.1. Problemas Encontrados.....	15
6.2. Aprendizaje.....	16



1. INTRODUCCIÓN

Este proyecto consiste en un sistema de gestión de activos informáticos e incidencias desarrollado en Python. El objetivo principal es llevar un control centralizado de su inventario de equipos informáticos y registrar las incidencias técnicas de cada activo.

La aplicación implementa una arquitectura por capas que separa la lógica de negocio, el acceso a datos y la presentación. Utiliza SQLite como sistema de gestión de bases de datos por su simplicidad. La interfaz gráfica está hecha con CustomTkinter, una librería que proporciona componentes visuales personalizables y una apariencia profesional.

2. ARQUITECTURA DEL SISTEMA

2.1. Organización por Capas

El proyecto sigue una arquitectura de tres capas que facilitan el mantenimiento y la escalabilidad del código. Esta separación permite que los cambios en una capa no afecten a las demás, cumpliendo con el principio de máxima modularidad.

La capa de datos está compuesta por el módulo database que gestiona la conexión con SQLite y la creación de tablas, y los repositorios que contienen exclusivamente las consultas SQL. Ninguna otra parte del código puede ejecutar consultas SQL directamente, garantizando así que toda la interacción con la base de datos esté centralizada.

La capa de lógica de negocio está formada por los servicios, que contienen toda la lógica de validación y las reglas de negocio. Los servicios llaman a los repositorios para acceder a los datos y procesan la información antes de



devolverla a la interfaz. Por ejemplo, el servicio de activos valida que el código tenga el formato correcto antes de intentar insertar el activo en la base de datos.

La capa de presentación incluye todas las interfaces creadas con CustomTkinter. Estas pantallas llaman a los servicios cuando necesitan datos o quieren realizar operaciones, pero nunca acceden directamente a los repositorios ni a la base de datos.

2.2. Base de Datos

La base de datos se implementa con SQLite, almacenándose en un único archivo llamado DBActivos.db que se crea automáticamente en la carpeta db al ejecutar la aplicación por primera vez. SQLite es una base de datos relacional completa que no requiere servidor ni configuración, lo que la hace ideal para este proyecto.

Al arrancar la aplicación, se ejecutan automáticamente las funciones `crear_tablas` e `insertar_datos_prueba`. La primera crea las tablas `activos` e `incidencias` si no existen, mientras que la segunda inserta diez activos y diez incidencias de ejemplo solo si la base de datos está vacía. Esto permite que el usuario pueda empezar a probar la aplicación inmediatamente sin necesidad de introducir datos manualmente.

Un aspecto importante de la base de datos es la activación de las claves foráneas mediante `PRAGMA foreign_keys = ON`. SQLite tiene las claves foráneas desactivadas por defecto. En este proyecto, cada incidencia está vinculada a un activo mediante una clave foránea con `ON DELETE CASCADE`, lo que significa que al eliminar un activo se borran automáticamente todas sus incidencias asociadas.



2.3. Modelo de Datos

El modelo de datos se compone de dos tablas principales relacionadas entre sí. La tabla activos almacena la información de cada equipo informático del inventario, mientras que la tabla incidencias registra los problemas técnicos asociados a cada activo.

La tabla activos tiene como clave primaria `id_activo` que se genera automáticamente mediante `AUTOINCREMENT`. El campo código es de tipo `TEXT` y tiene las restricciones `UNIQUE` y `NOT NULL` porque cada activo debe tener un código único e identificador. Los demás campos como tipo, marca, modelo, `num_serie`, `ubicacion`, `fecha_alta` y `estado` son todos de tipo `TEXT`. El campo código con la restricción `UNIQUE` es importante porque previene duplicados a nivel de base de datos.

La tabla incidencias utiliza `id_activo` como clave foránea que referencia a `activos(id_activo)` con `ON DELETE CASCADE`. Esto significa que la relación entre tablas está garantizada por la base de datos misma. El campo `id_activo` tiene la restricción `NOT NULL` porque no tiene sentido tener una incidencia que no esté asociada a ningún activo. Los campos `fecha_apertura`, `prioridad`, `categoria`, `descripcion`, `estado` y `tecnico_asignado` son todos `TEXT` por la misma razón que en la tabla activos.

3. FUNCIONALIDADES

3.1. Gestión de Activos

El sistema permite realizar todas las operaciones CRUD sobre los activos informáticos. La creación de activos se realiza mediante un formulario que solicita código, tipo, marca, modelo, número de serie, ubicación y estado. El



código debe seguir el formato ACT-XXXX donde X son cuatro dígitos, mientras que el número de serie debe tener el formato SNXXXXXX con seis dígitos. Estas validaciones se realizan mediante expresiones regulares en la capa de servicios antes de intentar insertar en la base de datos.

La búsqueda de activos funciona en tiempo real mediante una barra de búsqueda que filtra la tabla mientras el usuario escribe. El filtrado se implementa mediante una consulta SQL con LIKE que busca coincidencias en todos los campos del activo: código, marca, modelo, tipo, ubicación, número de serie y estado. Esto permite encontrar rápidamente cualquier activo escribiendo parte de cualquiera de sus datos.

La modificación de activos abre un formulario prellenado con los datos actuales del activo seleccionado. El ID y la fecha de alta no son modificables porque son datos que no deben cambiar una vez creado el activo. Todos los demás campos pueden editarse, y al guardar se ejecuta una validación completa igual que en la creación, incluyendo la comprobación de que el código no esté duplicado en otro activo diferente.

La eliminación de activos solicita confirmación antes de proceder porque es una acción destructiva. Cuando se elimina un activo, primero se borran todas sus incidencias asociadas y luego el activo mismo. Esto se hace en dos pasos separados en el repositorio para evitar problemas de integridad referencial, aunque con ON DELETE CASCADE la base de datos lo haría automáticamente.

3.2. Gestión de Incidencias

La gestión de incidencias permite crear nuevas incidencias asociadas a activos existentes. El formulario de creación incluye un selector de activo que muestra tanto el ID como el código para facilitar la identificación. Los campos obligatorios



son activo asociado, prioridad, categoría, descripción, técnico asignado y estado. La fecha de apertura se genera automáticamente con la fecha y hora actual del sistema.

A diferencia de los activos, las incidencias no tienen búsqueda por texto porque según el enunciado del proyecto, los activos deben tener búsqueda mientras que las incidencias deben tener filtros específicos. Por ello se implementaron filtros combinados por estado, prioridad y categoría aplicando múltiples criterios simultáneamente.

El cambio de estado de incidencias se realiza mediante un formulario específico que solo permite modificar el estado. Esto se diseñó así porque es la operación más común que realizan los técnicos. El formulario muestra el estado actual preseleccionado y permite cambiarlo entre Abierta, En Proceso y Cerrada. Otros campos como la descripción o la prioridad no se modifican porque normalmente se establecen al crear la incidencia y no deberían cambiar.

La eliminación de incidencias también requiere confirmación y solo está disponible para administradores. Los técnicos no pueden eliminar incidencias, solo cambiar su estado.

4. EXTRAS IMPLEMENTADOS

4.1. Extra 6: Sistema de Usuarios y Roles

El sistema implementa dos roles de usuario diferentes con permisos claramente diferenciados. El rol de Administrador tiene acceso completo a todas las funcionalidades del sistema, mientras que el rol de Técnico tiene acceso limitado a operaciones específicas.



Los administradores pueden crear, modificar y eliminar activos sin restricciones. También pueden crear, modificar estado y eliminar incidencias. Además tienen acceso exclusivo al menú de Importar/Exportar que les permite exportar datos filtrados e importar activos desde archivos CSV. Este acceso exclusivo se controla mostrando u ocultando el botón del menú según el rol.

Los técnicos pueden modificar activos pero no eliminarlos ni crearlos. Esta decisión de diseño se tomó porque los técnicos necesitan poder actualizar información como la ubicación o el estado de los equipos, pero no deberían tener capacidad para añadir o borrar activos del inventario. En cuanto a incidencias, los técnicos pueden ver todas las incidencias y cambiar su estado, que es su función principal, pero no pueden crear nuevas incidencias ni eliminarlas.

La autenticación se realiza mediante un sistema simple de login que valida usuario y contraseña contra un diccionario predefinido. Aunque en una aplicación real esto debería estar en base de datos con contraseñas hasheadas, para este proyecto de prácticas se optó por un diccionario en memoria que se reinicia cada vez que se ejecuta la aplicación. El sistema viene con dos usuarios de prueba: brian con rol Administrador y valen con rol Técnico, ambos con contraseña brian123 y valen123 respectivamente.

4.2. Extra 7: Validaciones Avanzadas

Las validaciones avanzadas se implementaron en múltiples capas para garantizar la integridad de los datos. La primera capa de validación está en la interfaz gráfica, donde se comprueba que todos los campos obligatorios estén completos antes de enviar el formulario. Esto evita llamadas innecesarias al servicio y da feedback inmediato al usuario.



La segunda capa está en los servicios, donde se validan los formatos mediante expresiones regulares. El código del activo debe cumplir el patrón ACT-XXXX con exactamente cuatro dígitos, y el número de serie debe cumplir SNXXXXXX con exactamente seis dígitos. Si alguna validación falla, el servicio devuelve False y un mensaje de error descriptivo que se muestra al usuario.

La tercera capa de validación comprueba duplicados antes de intentar insertarlos en la base de datos. Aunque la base de datos tiene una restricción UNIQUE en el campo código, validar antes permite devolver un mensaje de error en lugar del mensaje técnico de error de SQLite. Esta validación se hace obteniendo todos los activos y recorriendo la lista buscando coincidencias en el código.

Además de estas validaciones explícitas, la base de datos tiene sus propias restricciones que actúan como última línea de defensa. El campo código es UNIQUE NOT NULL y el campo id_activo de incidencias es NOT NULL con clave foránea. Estas restricciones a nivel de base de datos garantizan la integridad incluso si las validaciones en código fallaran por algún motivo.

4.3. Extra 9: Exportación Avanzada

La exportación avanzada se implementó creando pantallas específicas para exportar activos e incidencias con múltiples opciones de filtrado. En lugar de exportar siempre todos los datos, el usuario puede definir qué información quiere exportar aplicando diferentes criterios.

Para la exportación de activos, se implementaron filtros por estado y por tipo de equipo. El usuario selecciona los valores deseados en selectores desplegables y solo se exportan los activos que cumplan ambos criterios. Por ejemplo, puede exportar únicamente los portátiles que estén operativos. El sistema valida que



haya al menos un activo que cumpla los filtros antes de permitir guardar el archivo.

La exportación de incidencias añade complejidad con el filtro de rango de fechas. El usuario puede introducir una fecha desde y una fecha hasta, y el sistema extrae la parte de fecha de `fecha_apertura`, la convierte a objeto `datetime` y comprueba si está dentro del rango. Este proceso requiere manejo de excepciones porque el usuario podría introducir un formato de fecha incorrecto.

Ambas exportaciones permiten elegir entre formato CSV y JSON. El formato CSV se escribe usando el módulo `csv` de Python que maneja correctamente el escapado de caracteres especiales. El formato JSON se escribe usando `json.dump` con los parámetros `indent=4` para formato legible y `ensure_ascii=False` para preservar caracteres especiales como las tildes.

4.4. Extra 10: Mejoras de Usabilidad

Las mejoras de usabilidad se implementaron en varios aspectos de la interfaz para hacer la aplicación más intuitiva y eficiente. Los atajos de teclado permiten realizar acciones comunes sin usar el ratón, mejorando la velocidad de uso.

En la pantalla de activos se implementaron tres atajos: `Ctrl+N` para crear un nuevo activo, `Ctrl+M` para modificar el seleccionado y `Ctrl+E` para eliminar. Estos atajos se vinculan mediante el método `bind` de Tkinter que captura los eventos de teclado y ejecuta la función correspondiente. Los atajos solo funcionan cuando la pantalla de activos está visible porque se vinculan al objeto `app` que se limpia al cambiar de pantalla.



Las confirmaciones antes de eliminar previenen borrados accidentales. Cuando el usuario pulsa Eliminar, se muestra un diálogo `messagebox.askyesno` que requiere confirmación explícita. Solo si el usuario confirma se procede con la eliminación. Este patrón se aplica tanto en activos como en incidencias.

La interfaz es homogénea en todas las pantallas usando las mismas fuentes, tamaños y colores. Se definieron constantes como `ANCHO_BTN`, `ALTO_BTN` y `FONT_BTN` que se utilizan consistentemente en todos los botones. Esto da una apariencia profesional y facilita el mantenimiento porque cambiar estos valores en un sitio los cambia en toda la aplicación.

Los mensajes de estado informativos comunican claramente al usuario qué ha pasado. Cuando se crea un activo con éxito se muestra "Activo creado correctamente", cuando hay un error se muestra el motivo específico del error. Estos mensajes se implementan con `messagebox.showinfo` para éxitos y `messagebox.showerror` para errores, diferenciándolos visualmente.

5. INSTRUCCIONES DE USO

5.1. Instalación

Para instalar y ejecutar la aplicación, primero se debe verificar que Python 3.10 o superior esté instalado en el sistema. Esto puede comprobarse abriendo una terminal y ejecutando `python --version`.

Una vez confirmado Python, se debe instalar la única dependencia externa del proyecto que es CustomTkinter. Esto se hace abriendo una terminal en la carpeta del proyecto y ejecutando `pip install customtkinter`.

Con las dependencias instaladas, se navega a la carpeta UI del proyecto donde están los archivos de interfaz. Desde esta carpeta se ejecuta `python main_ui.py`



para iniciar la aplicación. La primera vez que se ejecuta, la aplicación crea automáticamente la base de datos DBActivos.db en la carpeta db e inserta los datos de prueba. Las siguientes ejecuciones simplemente abren la aplicación con los datos ya existentes.

Si se desea empezar con una base de datos limpia, basta con eliminar el archivo DBActivos.db de la carpeta db. La próxima vez que se ejecute la aplicación, se creará una nueva base de datos vacía y se insertarán los datos de prueba de nuevo.

5.2. Usuario Administrador

El administrador tiene acceso completo a todas las funcionalidades del sistema. Para acceder como administrador, se introduce el usuario brian y la contraseña brian123 en la pantalla de login. Al iniciar sesión, aparece el menú principal con tres opciones: Activos, Incidencias e Importar/Exportar.

En la gestión de activos, el administrador puede crear nuevos activos pulsando el botón Añadir. Para modificar un activo existente, el administrador lo selecciona en la tabla y pulsa Modificar. Para eliminar, se selecciona el activo y se pulsa Eliminar, confirmando en el diálogo que aparece.

En incidencias, el proceso es similar pero con filtros en lugar de búsqueda. Se seleccionan los criterios de estado, prioridad y categoría en los selectores, y se pulsa Aplicar Filtros para refinar los resultados. Para crear una incidencia se pulsa Crear Incidencia y se rellena el formulario completo. El activo asociado se elige de una lista que muestra ID y código. Para cambiar el estado de una incidencia se selecciona en la tabla, se pulsa Cambiar Estado y se elige el nuevo estado.



El menú Importar/Exportar permite exportar activos o incidencias a CSV o JSON con filtros opcionales. Se seleccionan los filtros deseados, se elige el formato, se pulsa Exportar y se indica dónde guardar el archivo. Para importar activos, se pulsa Importar Activos desde CSV, se selecciona el archivo, y el sistema procesa cada fila mostrando al final cuántos se importaron correctamente y cuántos fallaron.

5.3. Usuario Técnico

El técnico tiene acceso limitado para las tareas que normalmente realiza. Para acceder como técnico se usa el usuario valen y la contraseña valen123. El menú principal solo muestra dos opciones: Activos e Incidencias, sin acceso a importar/exportar.

En activos, el técnico solo ve el botón Modificar. No puede crear activos nuevos ni eliminar los existentes. Puede modificar cualquier campo excepto el ID y la fecha de alta, lo cual es útil para actualizar ubicaciones o cambiar el estado.

En incidencias, el técnico solo ve el botón Cambiar Estado. No puede crear incidencias nuevas porque normalmente las crean los usuarios o administradores al reportar problemas. Su función es actualizar el estado de las incidencias existentes conforme las va procesando: de Abierta a En Proceso cuando empieza a trabajar en ella, y de En Proceso a Cerrada cuando la resuelve.

Los filtros de incidencias funcionan igual para técnicos que para administradores, permitiéndoles encontrar rápidamente las incidencias que les interesan.

6. CONCLUSIONES



6.1. Problemas Encontrados

Durante el desarrollo del proyecto surgieron varios problemas técnicos que requirieron investigación y prueba de diferentes soluciones. El primero y más complejo fue el de los imports circulares entre pantallas. Inicialmente todas las pantallas importaban `menu_principal` al inicio del archivo, y `menu_principal` importaba todas las pantallas. Python no podía resolver estas dependencias circulares y lanzaba `ImportError`.

La solución requirió entender cómo Python carga los módulos. Mover los imports dentro de las funciones resolvió el problema porque el import sólo se ejecuta cuando se llama a la función, momento en que todos los módulos ya están cargados.

El cuarto problema fue el bloqueo de la base de datos al eliminar activos que tenían incidencias asociadas. Inicialmente la tabla incidencias tenía la clave foránea sin la cláusula `ON DELETE CASCADE`. Cuando se intentaba eliminar un activo que tenía incidencias vinculadas. Esto causaba que la transacción fallara y dejará la base de datos en un estado bloqueado temporalmente. La solución fue añadir `ON DELETE CASCADE` a la definición de la clave foránea en la tabla incidencias. Aunque se mantuvo el código que elimina manualmente las incidencias primero por seguridad.

El tercer problema menos grave fue con la carpeta de exportación. Inicialmente el botón de exportar creaba una carpeta `exports/` en la carpeta actual, que al ejecutar desde `UI/` acababa siendo `UI/exports/`. Intentar calcular la ruta al directorio raíz del proyecto con `os.path.dirname` y `os.path.join` resultó complicado por cómo Python maneja las rutas relativas.



La solución final fue simplemente no crear carpeta automáticamente, sino dejar que el usuario elija dónde guardar el archivo mediante `filedialog.asksaveasfilename`. Esto es más flexible porque el usuario puede guardar donde quiera, y elimina la complejidad de gestionar rutas que dependían de desde dónde se ejecutaba la aplicación.

6.2. Aprendizaje

Este proyecto permitió aplicar y consolidar conocimientos de múltiples áreas del desarrollo de software. El trabajo con SQLite enseñó la importancia de las restricciones a nivel de base de datos como UNIQUE, NOT NULL y FOREIGN KEY. Aunque validar en código es importante para la experiencia de usuario.

La arquitectura por capas demostró su valor conforme el proyecto crecía. Al principio parecía complicado tener repositorios, servicios e interfaces separados. Pero cuando hubo que cambiar cómo funcionaba la validación de duplicados, sólo se modificó el servicio sin tocar la interfaz ni el repositorio. Esta separación de responsabilidades hizo el mantenimiento mucho más simple.

El desarrollo de interfaces gráficas con CustomTkinter mostró la importancia de la consistencia visual. Definir constantes para tamaños de fuentes y botones al principio permitió cambiarlos fácilmente en toda la aplicación. También se aprendió que la interfaz debe ser lo más simple posible, delegando toda la lógica compleja a las capas inferiores.

En general, el proyecto cumplió sus objetivos de demostrar comprensión de bases de datos relacionales, arquitectura por capas, validación de datos y desarrollo de interfaces gráficas. Los extras implementados añadieron funcionalidad real útil más allá de los requisitos mínimos. El código resultante es mantenible, está bien estructurado y funciona correctamente.

